Memory allocation

Just like in a dictionary, the memory is allocated and grows linearly based on the number of keys added. However, in an ordered dictionary, nearly 50% more memory is used to preserve the order of the items. Here's the code:

```
from collections import OrderedDict
import sys

ordd = OrderedDict()
d = {}

for x in range(15):
    ordd[x] = x
    d[x] = x
    print("ordered dict:", sys.getsizeof(ordd), "\t",
"dict:", sys.getsizeof(d))

'''
Output:
-------
ordered dict: 392       dict: 232
ordered dict: 424       dict: 232
ordered dict: 456       dict: 232
ordered dict: 488       dict: 232
ordered dict: 520       dict: 232
ordered dict: 744       dict: 360
ordered dict: 776       dict: 360
ordered dict: 808       dict: 360
ordered dict: 840       dict: 360
ordered dict: 872       dict: 360
ordered dict: 1312      dict: 640
ordered dict: 1344      dict: 640
ordered dict: 1376      dict: 640
ordered dict: 1408      dict: 640
ordered dict: 1440      dict: 640
```

You can go to *https://lerner.co.il/2019/05/12/python-dicts-and-memory-usage/* for further details.

Methods

The move_to_end method is used to move an existing key of the dictionary either to the end or to the beginning. For example, from collections import OrderedDict:

```
ord_dict = OrderedDict().fromkeys('OSFY')
print("Original Dictionary")
print(ord_dict)

 # Move the key to  end
ord_dict.move_to_end('O')
print("\nAfter moving key 'O' to end of dictionary :")
print(ord_dict)

 # Move the key to beginning
ord_dict.move_to_end('F', last = False)
print("\nAfter moving Key in the Beginning :")
print(ord_dict)

Output:
Original Dictionary
OrderedDict([('O', None), ('S', None), ('F', None), ('Y',
None)])

After moving key 'G' to end of dictionary :
OrderedDict([('S', None), ('F', None), ('Y', None), ('O',
None)])

After moving Key in the Beginning :
OrderedDict([('F', None), ('S', None), ('Y', None), ('O',
None)])
```

In the reversed method, reverse the order of the dict keys stored. For example, from collections import OrderedDict:

```
x = {'d':'four', 'e':'five', 'a':'one'}

sample_reversed_dict = OrderedDict(reversed(list(x.items())))
print("The reversed order dictionary : " + str(sample_
reversed_dict))

Output:
The reversed order dictionary : OrderedDict([('a', 'one'),
('e', 'five'), ('d', 'four')])
```

Counter

Hold the count for each key. Adding a new key will create a single count and updating the existing key will increase the count. Here's an example:

```
from collections import Counter

counter_dict_sample = Counter("helloworld")
print(counter_dict_sample)

Output:
Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, 'w': 1, 'r': 1, 'd':
1})
```

ChainMap

ChainMap holds the list of dictionaries, where key search is performed based on the order of the dictionaries stored. For example, interpreters use nested scopes, where each mapping represents a scope context.